2.Instructions and Instruction Set Architecture

Contents

2.1 x86-64 Architecture

- It is an extended version of the x86 architecture originally developed by AMD.
- It supports 64-bit computing.
- It is fully compatible with its predecessors (16-bit and 32-bit architectures).
- It is a register-memory architecture.

2.2 General-Purpose Integer Registers

- A register is a temporary storage in the processor used for storing intermediate values while conducting calculation.
- X86-64 defines 16 integer registers.
- Each register is 64 bits wide.
- The range of integer values can be stored in each register is shown in Table 2.1 and 2.2.

2.2 General-Purpose Integer Registers

Table 2.1: Range of unsigned 64-bit integer values

Type	Binary value	Unsigned decimal value
Minimum value	$\underbrace{000\ldots000}_{64\text{ bits}}$	0
Maximum value	$\underbrace{111\ldots111}_{64\text{ bits}}$	$2^{64}-1$

Table 2.2: Range of signed 64-bit integer values

Type	Binary value	Signed decimal value
Minimum value	$1\underbrace{00\ldots000}_{63\text{ bits}}$	-2^{63}
Maximum value	$0\underbrace{11\ldots111}_{63\text{ bits}}$	$+2^{63}-1$

2.2 General-Purpose Integer Registers

- Interpreting signed and unsigned integer values (using two's complement method)

- Unsigned integer value: 1000…0001

  $1000\ldots0001 = (1*2^{63})+(0*2^{62})+\ldots+(0*2^{1})+(1*2^{0}) = 2^{63}+1$

- Signed integer value: 1000…0001

  $1000\ldots0001 = (1*-2^{63})+(0*2^{62})+\ldots+(0*2^{1})+(1*2^{0}) = -2^{63}+1$

2.2 General-Purpose Integer Registers

- We refer to each register by its name.
- Table 2.3 shows the register names for x86-64.

%rax	%rbx	%rcx	%rdx
%rbp	%rsp	%rsi	%rdi
%r8	%r9	%r10	%r11
%r12	%r13	%r14	%r15

From C.Nattee, Lecture Note of CSS224 Computer Architectures, 2017, Chapter 2.

2.3 Assembly Language

- An assembly language is a way to represent a sequence of machine instructions in a human-friendly form.

- To execute an assembly program, we need to have an assembler to translate the instructions into the machine code.

2.3 Assembly Language

- Here are how to write assembly instructions:

Assembly Instruction	Meaning
`OP B, A`	`A = A op B`
`OP A`	`A = op A`

where OP = an operation, A and B = operands which can be registers, constants, and memory.

- A dollar sign ($) is used to specify an integer constant.
- For example,
  add $1,%rax → %rax = %rax + 1

2.3.1 Operands

- Three types of operands can be used in the assembly instructions.

1. Register is referred by its name.

2. Constant is any numerical literal preceded with a $.

3. Memory location can be referred to in several ways.
   - A way to refer to a memory location is called an addressing mode.

2.3.1 Operands

- Note:
  '0x' is a marker for a hexadecimal constant, and
  '0b' is a marker for a binary constant.

- Table 2.4: Example of Operands in Assembly Instructions

Type	Instruction	Meaning
Register	`add %rax, %rcx`	`%rcx += %rax`
Decimal constant	`add $4, %rcx`	`%rcx += 4`
Hexadecimal constant	`add $0xf, %rcx`	`%rcx += 15`
Binary constant	`add $0b1100, %rcx`	`%rcx += 12`
Absolute address	`add 1234, %rcx`	`%rcx += Mem[1234]`
Indirect address	`add (%rax), %rcx`	`%rcx += Mem[%rax]`
Indirect address	`add 4(%rax), %rcx`	`%rcx += Mem[%rax+4]`

From C.Nattee, Lecture Note of CSS224 Computer Architectures, 2017, Chapter 2.

2.3.1 Operands

- For example,

add $11,%rax	→	%rax = %rax + 11
add $0x11,%rax	→	% rax = %rax + 17
add $0b11,%rax	→	% rax = %rax + 3

2.3.2 Instructions

- Table 2.5 shows basic integer instructions in x86-64.
- Table 2.5 Integer Arithmetic Instruction 1

Operation	Instruction	Meaning
Copy	`mov S, D`	`D = S`
Add	`add S, D`	`D = D + S`
Subtract	`sub S, D`	`D = D - S`
Negation	`neg D`	`D = -D`
Multiply	`imul S, D`	`D = D * S` *(D must be a register.)*

Divide	`idiv S`	`%rax = %rdx:%rax / S`
		`%rdx = %rdx:%rax % S`
		(S must be a register.)

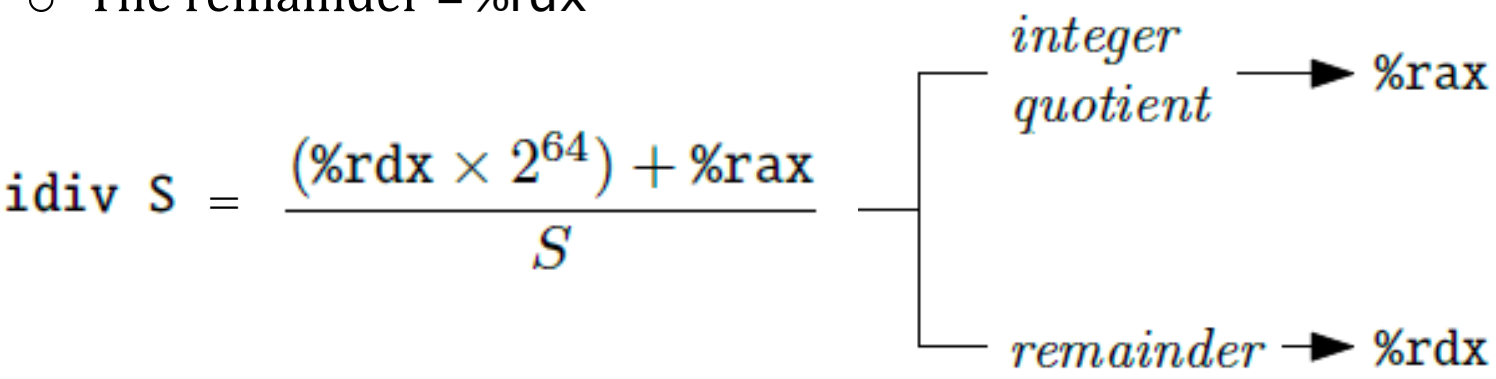
Convert	`cqto`	Extend `%rax` to `%rdx:%rax`
Increment	`inc D`	`D = D + 1`
Decrement	`dec D`	`D = D - 1`

From C.Nattee, Lecture Note of CSS224 Computer Architectures, 2017, Chapter 2.

2.3.1 Operands

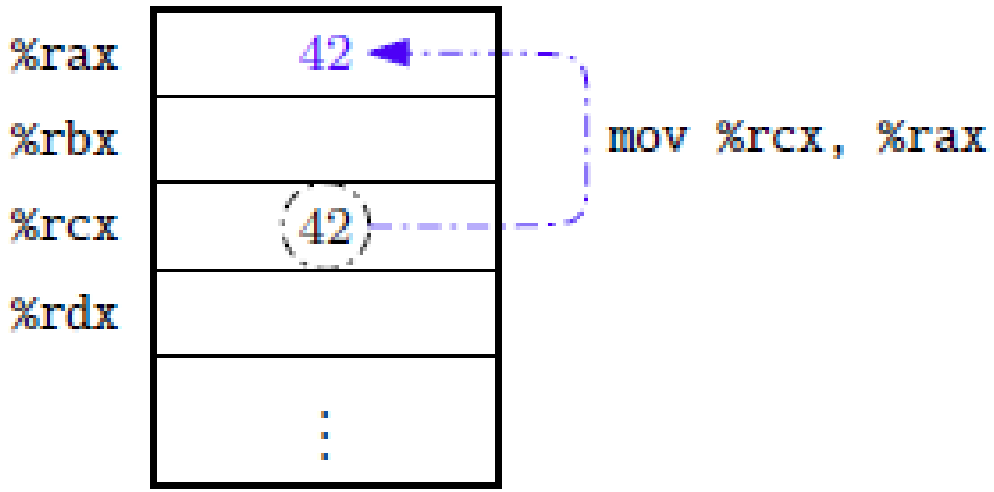
- An instruction idiv S divides a 128-bit value constructed by concatenating %rdx and %rax by a register S.

- Note: dividend = quotient * divisor + remainder
  - The dividend = %rdx:%rax
  - The divisor = S
  - The integer quotient = %rax
  - The remainder = %rdx



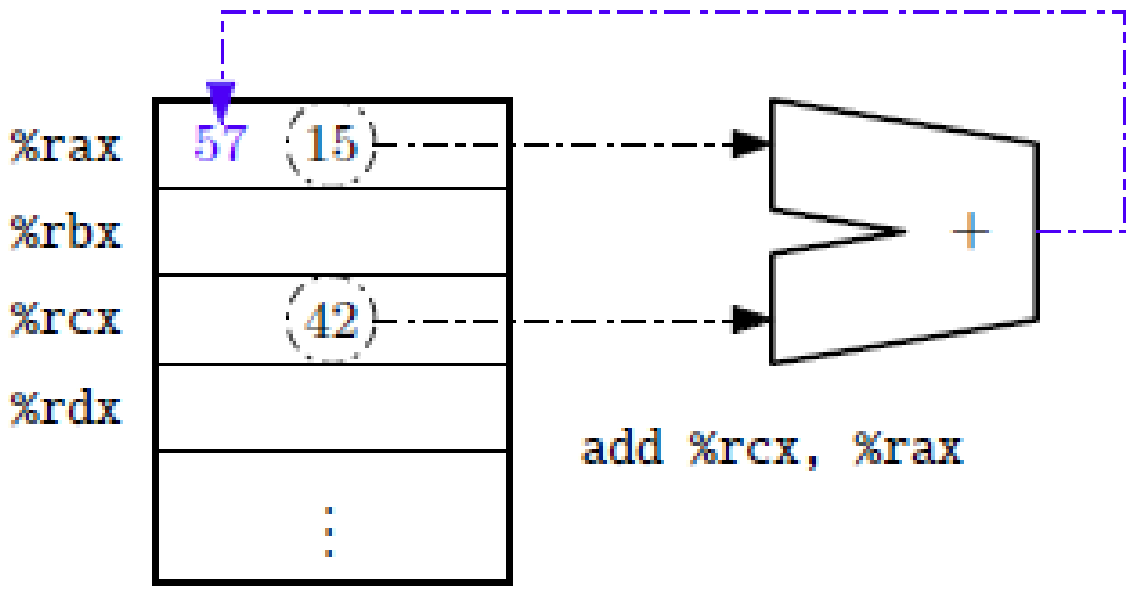
2.3.2 Instructions

- Instruction Execution Examples: mov %rcx,%rax



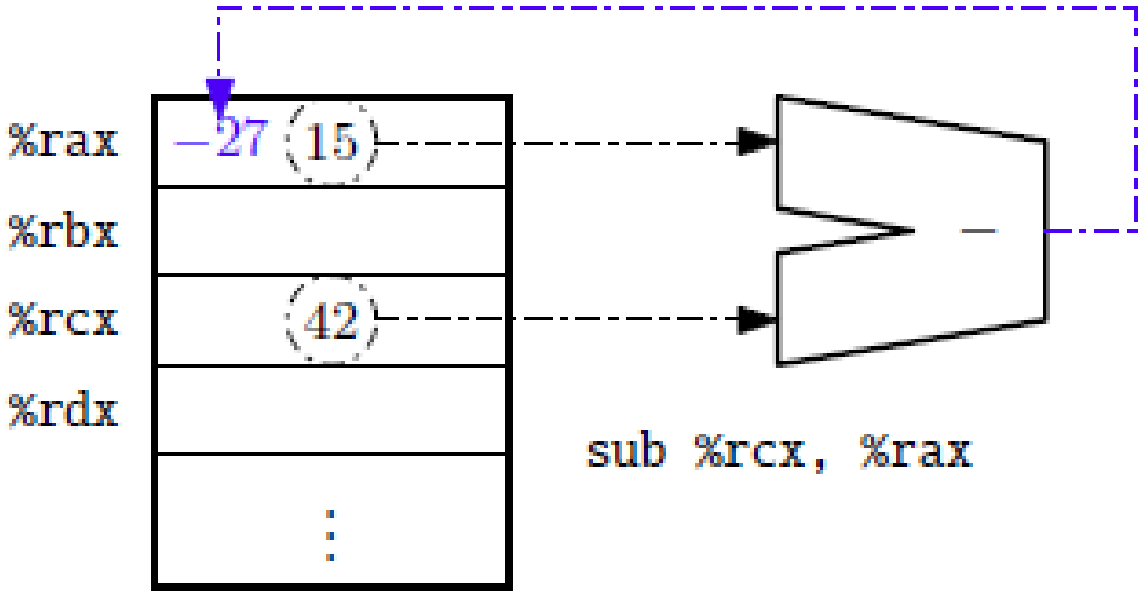
2.3.2 Instructions

- Instruction Execution Examples: add %rcx,%rax



2.3.2 Instructions

- Instruction Execution Examples: sub %rcx,%rax



Exercise

- Let %rax = 1, %rcx = 1024, and %rdx = 64. Wrtie the output of the following instructions.

1. add %rcx,%rax

2. neg %rax

3. idiv %rcx

2.3.2 Instructions

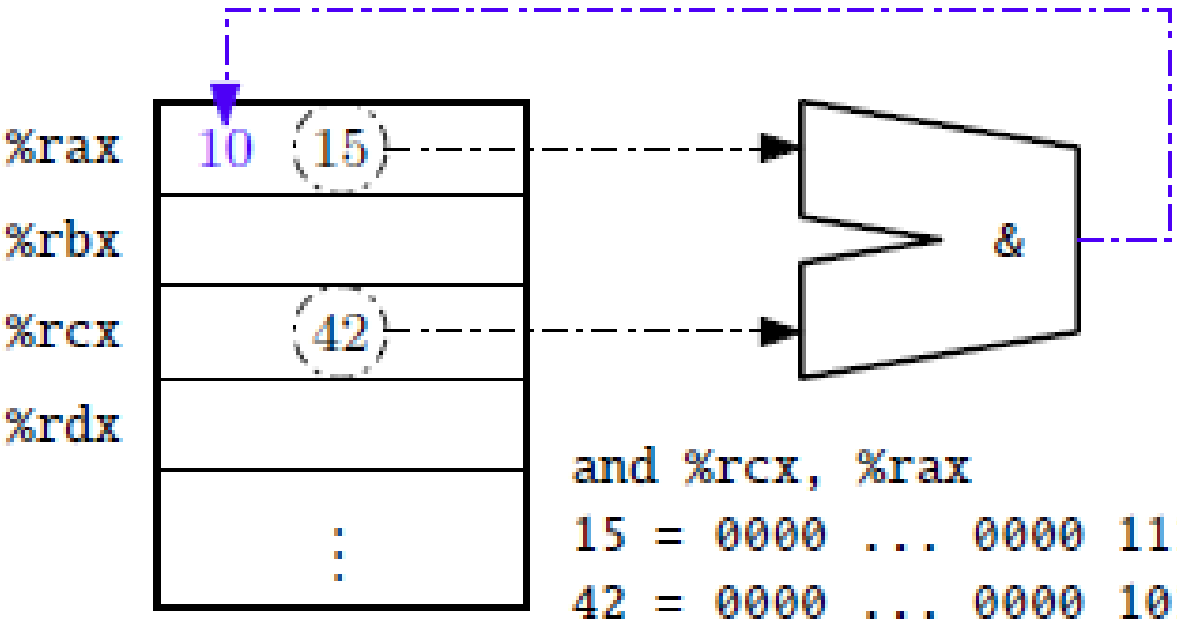
- Table 2.6 shows basic integer instructions in x86-64.
- Table 2.6 Integer Arithmetic Instruction 2

Operation	Instruction	Meaning
Bitwise And	`and S, D`	`D = D & S`
Bitwise Or	`or S, D`	`D = D | S`
Bitwise Xor	`xor S, D`	`D = D ^ S`
Bitwise Not	`not D`	`D = ~D`
Left shift	`shl S, D`	`D = D << S`
Unsigned right shift	`shr S, D`	`D = D >>> S`
Signed right shift	`sar S, D`	`D = D >> S`
Push	`push S`	Push S on the system stack
Pop	`pop D`	Pop from the system stack into D

From C.Nattee, Lecture Note of CSS224 Computer Architectures, 2017, Chapter 2.

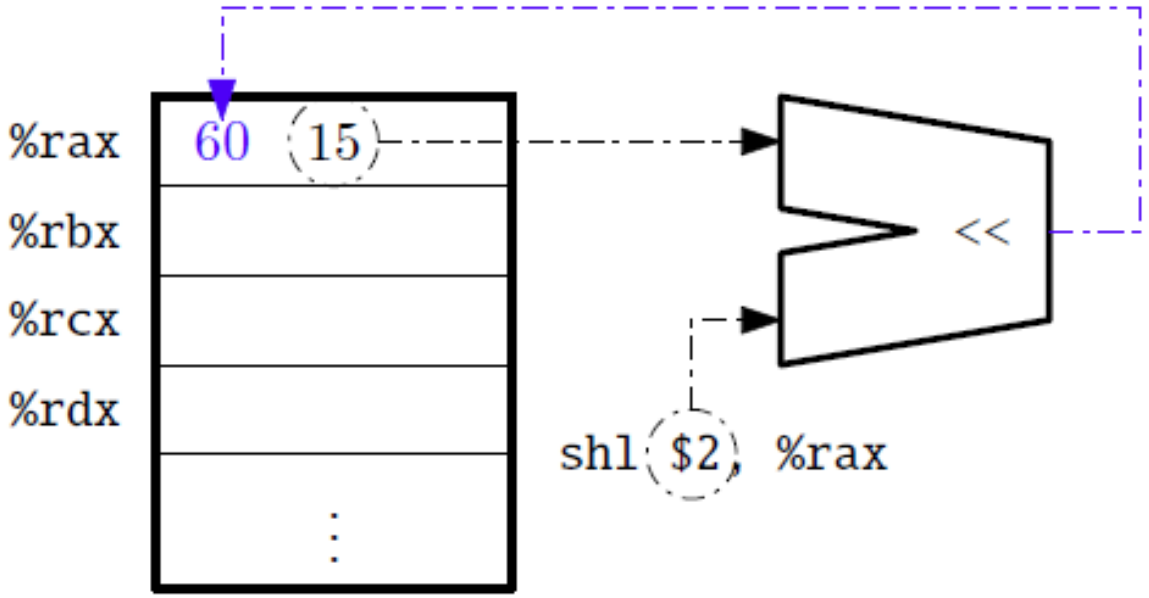
2.3.2 Instructions

- Instruction Execution Examples: and %rcx,%rax



2.3.2 Instructions

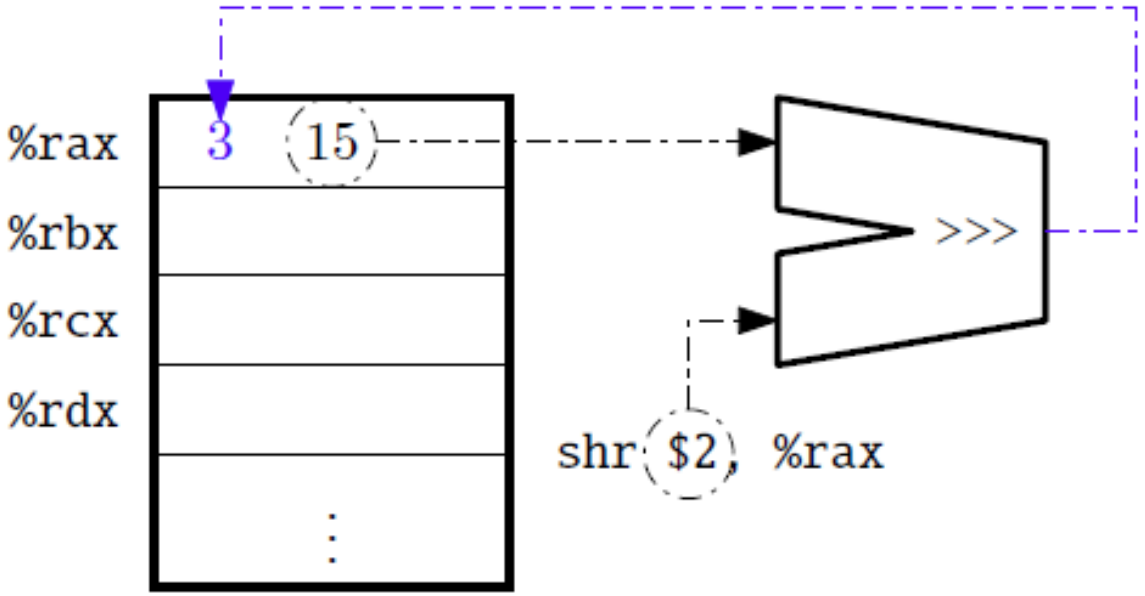
- Instruction Execution Examples: shl $2,%rax



15 << 2 = 0000 ... 0000 1111 << 2
= 0000 ... 0011 1100
= 60

2.3.2 Instructions

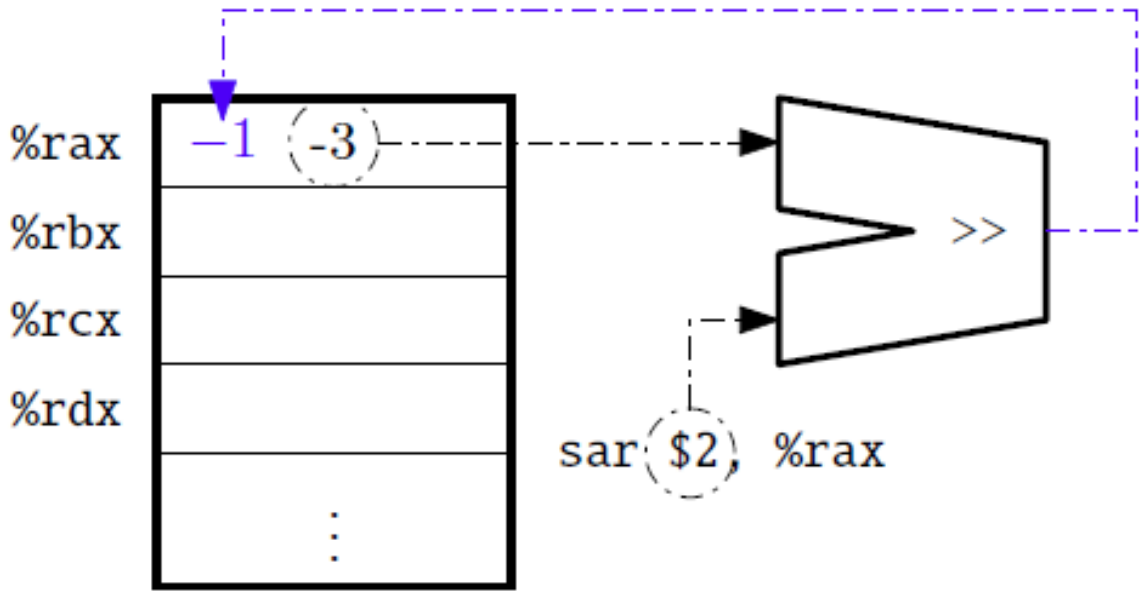
- Instruction Execution Examples: shr $2,%rax




```
15 >>> 2 = 0000 ... 0000 1111 >>> 2
          = 0000 ... 0000 0011
          = 3
```

2.3.2 Instructions

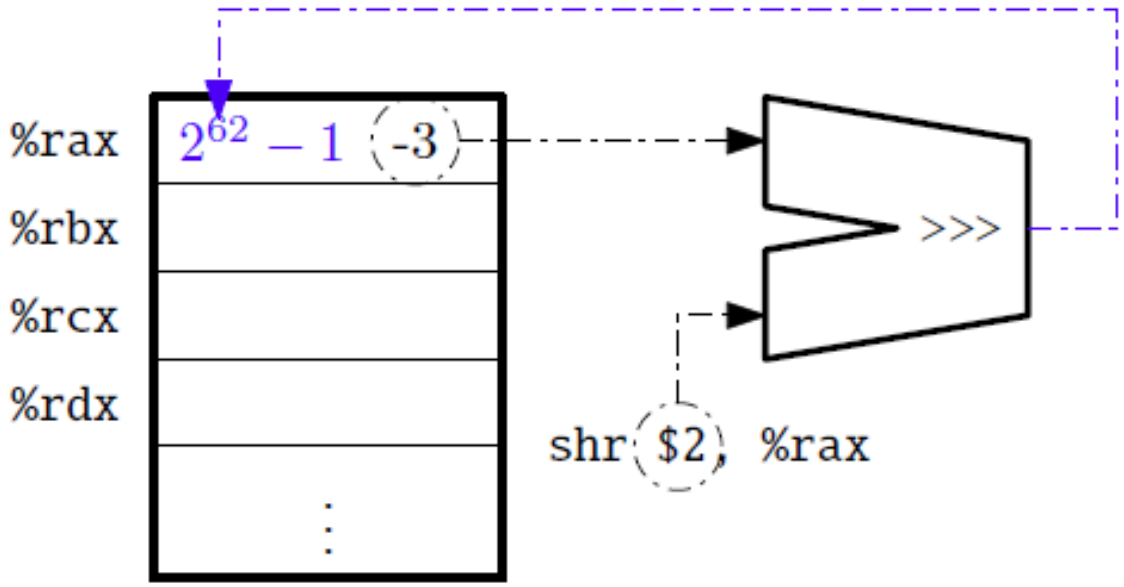
- Instruction Execution Examples: sar $2,%rax




```
-3 >> 2 = 1111 ... 1111 1101 >> 2
        = 1111 ... 1111 1111
        = -1
```

2.3.2 Instructions

- Instruction Execution Examples: shr $2,%rax



$$-3 >>> 2 = 1111 \ldots 1111\ 1101 >>> 2$$
$$= 0011 \ldots 1111\ 1111$$
$$= 2^{62} - 1$$

Exercise

- Let %rax = 10 and %rcx = 1024. Wrtie the output of the following instructions.

1. not %rax
2. or $-1,%rax
3. shl $4,%rcx
4. shr $4,%rcx

2.4 Assembly Program

- An assembly program mainly is composed of a sequence of assembly instructions.
- The processor do not understand the assembly instructions.
- An assembler is needed to translate assembly instructions into machine code.
- Each assembly instruction can be one-to-one translated into a machine instruction.

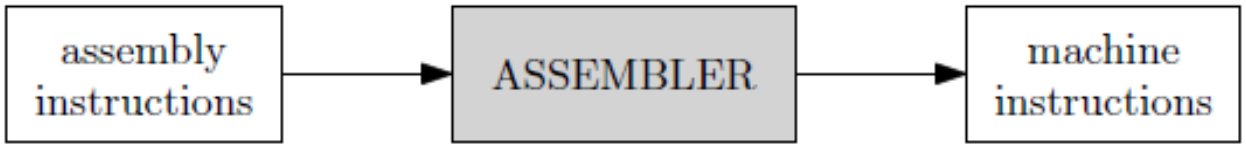


Figure 2.7: Assembler

From C.Nattee, Lecture Note of CSS224 Computer Architectures, 2017, Chapter 2.

2.4 Assembly Program

- Each assembly program also includes a number of assembler directives
  - To tell the assembler how to translate the program
  - To tell the assembler how to create an executable file.

- A directive always starts with a dot.
- Table 2.7 shows assembler directives.

Directive	Meaning
`.global`	Set a label as a global symbol
`.text`	Start a text section for a sequence of instructions, or read-only data
`.data`	Start a data setion for defining memory locations
`.quad`	Define a 64-bit integer location
`.asciz`	Define a null-terminated character string

From C.Nattee, Lecture Note of CSS224 Computer Architectures, 2017, Chapter 2.

2.4 Assembly Program

- An assembly program may include a number of labels.
- Each label is a number ended with a colon.
- It represents a memory address of instructions or constants.

2.4 Assembly Program

- Example2.2 An assembly program that output “Hello world”.

```
        .global main
        .text
main:
        mov     $msg, %rdi      # set address of msg as the 1st argument
        call    puts            # call 'puts' to display a string
        xor     %rax, %rax      # set %rax = 0 (return value)
        ret                     # return (end of program)
msg:    .asciz  "Hello, world"
```

- The above assembly program is equivalent to the following C program.

```
#include<stdio.h>

int main() {
  puts("Hello, world");
  return 0;
}
```

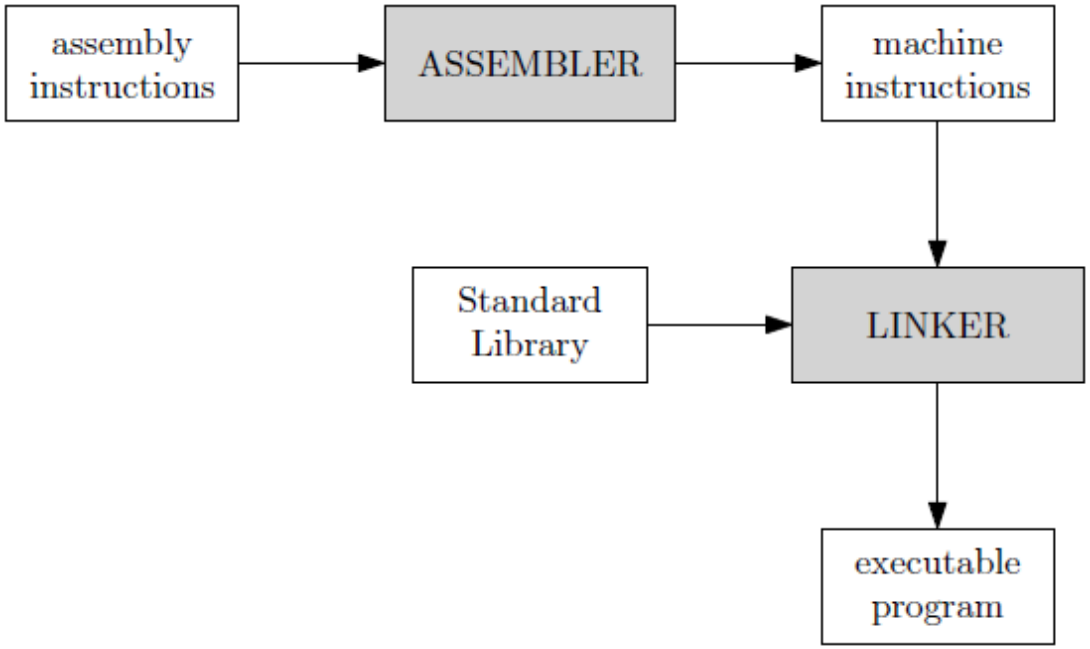
2.4 Assembly Program

- Here is the output obtaining from the assembler.

```
GAS LISTING hello.s                                     page 1


   1                            .global main
   2                            .text
   3                      main:
   4 0000 48C7C700                      mov     $msg, %rdi
   4      000000
   5 0007 E8000000                      call    puts
   5      00
   6 000c 4831C0                        xor     %rax, %rax
   7 000f C3                            ret
   8 0010 48656C6C      msg:    .asciz  "Hello, world"
   8      6F2C2077
   8      6F726C64
   8      00
GAS LISTING hello.s                                     page 2


DEFINED SYMBOLS
             hello.s:3      .text:0000000000000000 main
             hello.s:8      .text:0000000000000010 msg

UNDEFINED SYMBOLS
puts
```

2.4 Assembly Program

- The previous machine instructions still cannot be executed since a C standard library function (puts)
- We need to pass it to a linker to create an executable program.



From C.Nattee, Lecture Note of CSS224 Computer Architectures, 2017, Chapter 2.

2.5 Calling Convention

- We need to follow a standard scheme for calling C functions called 'calling convention'. Here are part of them:

  1.Pass parameters to function via registers in the following order:

  %rdi, %rsi, %rdx, %rcx, %r8, %r9.

  2.An integer value can be returned from a function via %rax.

  3.The following registers may be changed when we make a function call:

  %rax, %rcx, %rdx, %rsi, %rdi, %r8, %r9, %r10, %r11.

  4. We need to save the values of the following registers before we make any change:

  %rbx, %rbp, %r12, %r13, %r14, %r15.

2.5 Calling Convention

- We study the assembly language in order to understand how machine instructions work.
- We therefore do not write a stand-alone assembly program, but only implement functions that can be called from C programs in the assembly language.

2.5 Calling Convention

- Example 2.3 Implement a function sum(a, b, c) that accepts three integer values and returns the sum of them.

testsum.c

```
#include<stdio.h>

long sum(long a, long b, long c);

int main() {
  long x = sum(1, 2, 3);
  long y = sum(5, 3, 4);

  printf("sum(1, 2, 3) = %ld\n", x);
  printf("sum(5, 3, 4) = %ld\n", y);

  return 0;
}
```

2.5 Calling Convention

- Example 2.3 Implement a function sum(a, b, c) that accepts three integer values and returns the sum of them.

sum.s

```
# long sum(long a, long b, long c);
# according the calling convention:
#       a = %rdi
#       b = %rsi
#       c = %rdx

        .global sum
        .text
sum:
        mov     %rdi, %rax      # %rax = a
        add     %rsi, %rax      # %rax = %rax + b
        add     %rdx, %rax      # %rax = %rax + c
        ret                     # return %rax
```

2.6 Control Structures

- Each assembly program is just a sequence of instructions.
- No other structures are provided in the syntax of the language.
- We can control the instruction execution using two types of instructions, i.e., compare and jump instructions. Table 2.8 shows them.
- Control Structures

  2.6.1 Conditional statements

  2.6.2 Repetition statements

2.6 Control Structures

- Table 2.8 Compare and Jump instructions.

Instruction	Meaning
`cmp S, D`	Calculate `D - S` and set the *condition flags* without storing the result
`test S, D`	Calculate `D & S` and set the *condition flags* without storing the result; typically used to check the value in a register
`jmp L`	Unconditional jump to `L`
`je L`	Jump to `L` if the previous result $= 0$
`jne L`	Jump to `L` if the previous result $\neq 0$
`jg L`	Jump to `L` if the previous result > 0
`jge L`	Jump to `L` if the previous result ≥ 0
`jl L`	Jump to `L` if the previous result < 0
`jle L`	Jump to `L` if the previous result ≤ 0

2.6.1 Conditional Statements

- We can combine the compare and jump instructions in order to make the program work in the same manner as the condition statements (if–else).
- Example:

C Language

```
if (x > 100) {
    //do if true
}
else {
   //do if false
}
```

Assembly Language

```
     cmp $100, %rax
     jg  L1
     # do if false
     jmp LE
L1:  # do if true
LE:
```

2.6.1 Conditional Statements

- Example 2.5 Write a function in assembly that prints “>100” when an integer parameter is greater than 100, otherwise it prints “<=100”.

testg100.c

```
#include<stdio.h>

void g100(long a);

int main() {
  g100(45);
  g100(32090);
  g100(-190);
  g100(100);
  return 0;
}
```

2.6.1 Conditional Statements

- Example 2.5 (cont.)

g100.s

```
        .global g100
        .text
g100:
        cmp     $100, %rdi
        jg      LG
        mov     $les, %rdi
        call    puts
        jmp     EXIT
LG:     mov     $gre, %rdi
        call    puts
EXIT:   ret
les:    .asciz  "<=100"
gre:    .asciz  ">100"
```

2.6.2 Repetition Statements

- The compare and jump instructions can also be used to form repetition structures.
- Example 2.7 Write a function sumN that calculates 1+2+…+n where n is an argument.

sumN.c

```
long sumN(long N) {
  long i;
  long sum = 0;

  for(i=1; i<=N; i++) {
    sum += i;
  }

  return sum;
}
```

2.6.2 Repetition Statements

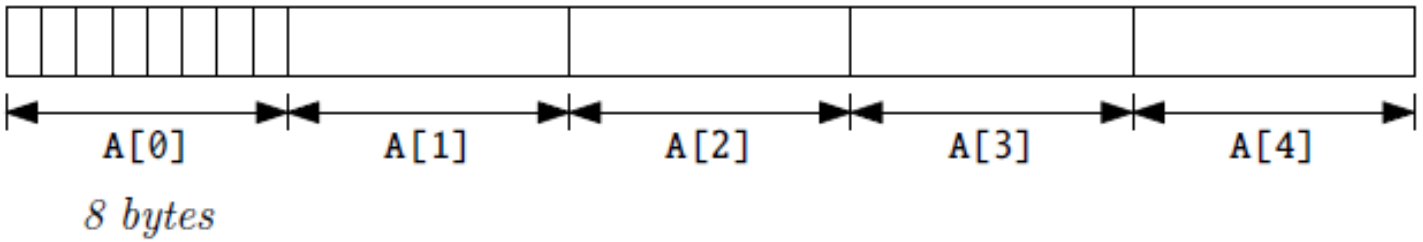
- Example 2.7 Write a function sumN that calculates 1+2+…+n where n is an argument.

sumN.s

```
        .global sumN
        .text
sumN:
        mov     $1, %rdx        # %rdx = i = 1
        xor     %rax, %rax      # %rax = sum = 0
loop:   cmp     %rdi, %rdx
        jg      done            # go to done if i>N or i-N>0
        add     %rdx, %rax      # sum += i
        add     $1, %rdx        # i++
        jmp     loop            # go to loop
done:
        ret
```

2.7 Arrays

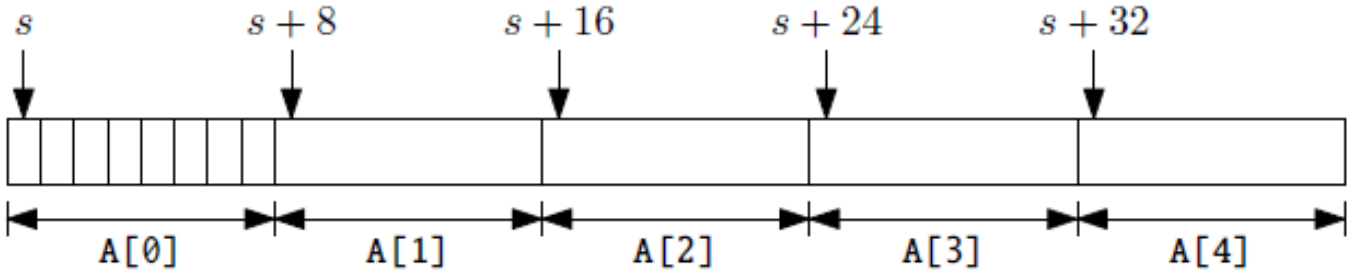
- An array is sequence of values of the same data type.
- It is stored in the memory as a consecutive block of memory.
- For example, a C statement long A[5]; defines an array which is composed of 5 elements. Each element is a 64-bit integer.



2.7.1 Accessing Array Elements

- Since each array is a consecutive block of data, we can calculate the address of an element in the array from:
  1. The starting address of the array,
  2. The size in bytes of an element, and
  3. The element index that we want to access.

- For example, suppose the starting address of the array A is s. The element A[3] can be found at s+8*3 = s+24.



2.7.1 Accessing Array Elements

- When we call a function in C and pass an array as an argument, the starting address is passed to the function.
- We can use the starting address to compute the addresses of elements.
- Example 2.8 Write a function sumArr(A,n) that computes the sum of the n elements in the array A.

testsumArr.c

```
#include<stdio.h>

long sumArr(long A[], long n);

int main() {
  long X[] = {1, 4, 5, 8, 3, 7, 6};
  printf("Output = %ld\n", sumArr(X, 7));
  return 0;
}
```

2.7.1 Accessing Array Elements

- When we call a function in C and pass an array as an argument, the starting address is passed to the function.
- We can use the starting address to compute the addresses of elements.
- Example 2.8 Write a function sumArr(A,n) that computes the sum of the n elements in the array A.

sumArr.s

```
        .global sumArr
        .text
sumArr:
        xor     %rax, %rax
        xor     %rcx, %rcx
loop:   cmp     %rsi, %rcx
        jge     done
        add     (%rdi), %rax
        inc     %rcx
        add     $8, %rdi
        jmp     loop
done:   ret
```

2.8 Example of Assembly Programs

Exercise

- Exercise 2.3 Implement a function add1000(a) that returns a + 1000. The function will be used by the following C program.

testadd1000.c

```
#include<stdio.h>

long add1000(long a);

int main() {
  long x;

  print("Enter a number: ");
  scanf("%ld", &x);                  // accept an integer from user

  print("Output = %ld\n", add1000(x));
  return 0;
}
```

Exercise

- Example 2.4 Implement a function ldgt(a) that returns the last digit of a.

testldgt.c

```
#include<stdio.h>

long ldgt(long a);

int main() {
  long x;
  printf("Enter a number: ");
  scanf("%ld", &x);
  printf("Output = %ld\n", ldgt(x));
  return 0;
}
```

Exercise

- Exercise 2.4 Implement a function exercise2(a,b,c) that returns c+2*(a+b)-(b+2*c) used by the following C program.

testexercise2.c

```
#include<stdio.h>

long exercise2(long a, long b, long c);

int main() {
  long x, y, z;
  printf("Enter three integers: ");
  scanf("%ld%ld%ld", &x, &y, &z);
  printf("Output = %ld\n", exercise2(x, y, z));
  return 0;
}
```

Exercise

- Exercise 2.5 Implement a function exercise3(a,b) that returns (a+b)%2 used by the following C program.

testexercise3.c

```
#include<stdio.h>

long exercise3(long a, long b);

int main() {
  long x, y;
  printf("Enter two numbers: ");
  scanf("%ld%ld", &x, &y);
  printf("Output = %ld\n", exercise3(x, y));
  return 0;
}
```

Exercise

- Exercise 2.6 Implement a function exercise4(a) that returns the second last digit of a.

testexercise4.c

```
#include<stdio.h>

long exercise4(long a);

int main() {
  long x;
  printf("Enter a number: ");
  scanf("%ld", &x);
  printf("Output = %ld\n", exercise4(x));
  return 0;
}
```

Exercise

- Example 2.6 Write a function in assembly that checks and displays if an integer parameter is odd or even.

testoddeven.c

```
#include<stdio.h>

void oddeven(long a);

int main() {
  oddeven(10);
  oddeven(11);
  return 0;
}
```

Exercise

- Exercise 2.7 Write a function maxoftwo(a,b) that returns the maximum value between a and b.

```
#include<stdio.h>

long maxoftwo(long a,long b);

int main() {
     long x;
     x = maxoftwo(5,10);
     printf("Output = %ld\n",x);
     return 0;
}
```

References

1. C.Nattee, Lecture Note of CSS224 Computer Architectures, 2017.
2. R.E. Bryant and D.R. O'Halloron, Computer Systems: A Programmer's Perspective, 2nd edition, 2011, Pearson/Prentice Hall, ISBN:978-0-13-610804-7.